Sanjivani College of Engineering

Department of Electronics and Computer Engineering

Academic Year: 2026–2027

EDUASSIST AI – AI BASED ACADEMIC ASSISTANT FOR STUDENT PERFORMANCE ENHANCEMENT

Project Report

Subject: Deep Learning

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology (B.Tech)

in

Electronics and Computer Engineering

Submitted by

Sr.	Name of Student	Roll Number
1	Sakshi Ramdas Zurale	UEC23F1069
2	Sakshi Vijay Thakare	UEC23F1064
3	Vrushali Sarangdhar Mhaismale	UEC21F1036

Under the guidance of

Dr. N. K. Darwante

CERTIFICATE

This is to certify that the project report entitled **"EduAssist AI – AI Based Academic Assistant for Student Performance Enhancement"** is a bonafide record of the work carried out by the following students:

Sr.	Name of Student	Roll Number	Signature
1	Sakshi Ramdas Zurale	UEC23F1069	
2	Sakshi Vijay Thakare	UEC23F1064	
3	Vrushali Sarangdhar Mhaismale	UEC21F1036	

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Electronics and Computer Engineering** at the Department of Electronics and Computer Engineering, Sanjivani College of Engineering, during the Academic Year **2026–2027**, under the guidance of Dr. N. K. Darwante, in the subject of Deep Learning.

Dr. N. K. Darwante **Project Guide**	Prof. S. S. Kulkarni **Course In-charge**	Dr. D. B. Kshirsagar **Head of Department**

Place: _____
Date: 12 October 2026

DECLARATION

We hereby declare that the project report entitled **"EduAssist AI – AI Based Academic Assistant for Student Performance Enhancement"**, submitted to the Department of Electronics and Computer Engineering, Sanjivani College of Engineering, in partial fulfillment of the requirements for the degree of Bachelor of Technology, is a record of original work carried out by us under the guidance of Dr. N. K. Darwante, as part of the subject Deep Learning.

The work has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma. All sources of information, tools, and published material used have been acknowledged in the references.

Sr.	Name of Student	Roll Number	Signature
1	Sakshi Ramdas Zurale	UEC23F1069	
2	Sakshi Vijay Thakare	UEC23F1064	
3	Vrushali Sarangdhar Mhaismale	UEC21F1036	

Place: _____
Date: 12 October 2026

e

ABSTRACT

Students usually see their academic information in separate pieces and after the fact, which leaves little time to act on a decline in performance. This report presents EduAssist AI, a cloud-oriented, web-based academic assistant that gives each student a single, student-specific view of their academic standing.

The system has a React front end, built with JavaScript, Vite, and CSS, and a Python FastAPI REST backend. The backend reads processed CSV student data with Pandas and applies two scikit-learn models: a Gradient Boosting Regressor that predicts a performance score and a Random Forest Classifier that assigns an academic risk class. It also generates Student Intelligence and Academic Insights, including recommendations and study guidance, and provides a chat endpoint in which a keyword-based intent classifier maps a student's question to an intent, reports an intent confidence, and answers from the student's own data. The assistant is rule-based and is not a large language model.

The report documents the architecture, technologies and algorithms, workflow, module structure, and REST endpoints, and presents eight system diagrams. No accuracy figures or other evaluation metrics are reported. Predictions are decision-support estimates and not guaranteed outcomes. Cloud deployment, authentication, and database storage are treated as future work.

Keywords: Deep Learning, Neural Networks, Supervised Learning, Student Performance Prediction, Academic Risk Classification, Intent Classification, Model Training and Inference

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 7
1.1 Background and Motivation 7
1.2 Problem Statement 7
1.3 Need for the Project 7
1.4 Project Objectives 7
1.5 Scope 7
1.6 Overview of EduAssist AI 7
CHAPTER 2: EXISTING SYSTEM AND PROBLEM ANALYSIS 9
2.1 Existing Approaches and Their Limitations 9
2.2 Problems Addressed by the Proposed System 9
2.3 Functional Requirements 9
2.4 Non-Functional Requirements 9
2.5 Feasibility Analysis 9
CHAPTER 3: PROPOSED SYSTEM AND ARCHITECTURE 11
3.1 Proposed Solution Overview 11
3.2 System Architecture 11
3.3 Frontend-Backend Interaction and API Communication 11
3.4 Data Flow 12
3.5 Machine-Learning Inference 12
3.6 Academic Insights Generation 12
3.7 Chat Processing and Response Generation 12
CHAPTER 4: TECHNOLOGIES AND ALGORITHMS 14
4.1 Technology Stack 14
4.2 Frontend and Backend Technologies 14
4.3 Gradient Boosting Regression 14
4.4 Random Forest Classification 14
4.5 Data Processing and Model Persistence 14
4.6 Keyword-Based Intent Classification 15
CHAPTER 5: SYSTEM DESIGN AND METHODOLOGY 16
5.1 Implemented Workflow 16
5.2 Machine-Learning Workflow 16
5.3 Academic Risk Classification 16
5.4 Performance Prediction 17
5.5 Chat Assistant Workflow 17
CHAPTER 6: IMPLEMENTATION 18
6.1 Frontend and Backend Structure 18
6.2 Components and Responsibilities 18
6.3 API Implementation 18
6.4 Model Loading, Data Processing, Insights, and Chat 19
6.5 Frontend-Backend Integration, Error Handling, and Validation 19
CHAPTER 7: TESTING AND RESULTS 20
7.1 Testing Methodology 20
7.2 Machine-Learning Functionality Testing 20
7.3 Recorded Output for a Sample Student 20
7.4 Output Analysis and Observed Limitations 21
CHAPTER 8: USER INTERFACE AND OUTPUT ANALYSIS 22
8.1 User Interface Design Principles 22
8.2 Dashboard and Student Overview 22
8.3 Performance Prediction and Risk Classification 22
8.4 Student Intelligence and Academic Insights 22
8.5 Chat Assistant Interface 22
8.6 API Documentation and Error Handling 22
CHAPTER 9: ADVANTAGES, LIMITATIONS AND APPLICATIONS 23
9.1 Advantages 23
9.2 Limitations 23
9.3 Applications 23
CHAPTER 10: FUTURE SCOPE AND CONCLUSION 24
10.1 Future Scope 24
10.2 Conclusion 24
REFERENCES 25

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Students receive academic information in fragments: internal marks, attendance, assignment scores, and examination results are usually reviewed separately and after the fact. By the time a decline in performance becomes visible, little time is left to correct it. Machine learning and educational data mining make it possible to move from retrospective reporting to early, student-specific analysis [1]. EduAssist AI is motivated by the need to give each student one place to see where they stand, where they may be heading, and what they can do about it.

1.2 Problem Statement

Conventional academic monitoring is manual, periodic, and generic. A student has no easy way to obtain a forward-looking estimate of performance, an indication of academic risk, personalised study guidance, or answers to academic questions that relate to their own data. This project designs and implements a web-based system that retrieves a student's academic data, applies machine-learning models to estimate future performance and classify academic risk, generates personalised insights, and answers academic questions through an interactive assistant.

1.3 Need for the Project

The project responds to three needs: early identification of students who may need academic support, centralised and understandable presentation of academic information, and an interactive interface through which students can ask questions about their own performance instead of interpreting raw numbers. Because the front end and the backend communicate through REST APIs, the project is also a practical exercise in cloud-oriented application design.

1.4 Project Objectives

1. To build a React-based web interface through which a student can access academic analysis using a student identifier.
2. To develop a FastAPI REST backend exposing student data, predictions, intelligence, insights, and chat through documented endpoints.
3. To predict a performance score with a Gradient Boosting Regressor and classify academic risk with a Random Forest Classifier.
4. To generate Student Intelligence and Academic Insights with recommendations and study guidance.
5. To provide a chat assistant that uses keyword-based intent classification and answers from the student's own data.

1.5 Scope

The scope covers the web front end, the REST backend, CSV-based student data processing, the two machine-learning models, the generation of intelligence and insights, and the keyword-based chat assistant. The system is **cloud-oriented**: it is organised as a client and a stateless REST service that can be hosted on cloud infrastructure. Deployment on a specific cloud provider is not part of the work reported here and is listed as future scope. Authentication, role-based access, and database storage are likewise outside the scope of the current implementation.

1.6 Overview of EduAssist AI

EduAssist AI is a web-based academic intelligence and assistance system. A student identifier is entered in the React front end, which calls the FastAPI backend.

CHAPTER 2: EXISTING SYSTEM AND PROBLEM ANALYSIS

2.1 Existing Approaches and Their Limitations

Academic monitoring in most institutions relies on mark sheets, spreadsheets, attendance registers, learning-management-system grade views, and periodic meetings with faculty mentors. These approaches record what has happened but seldom estimate what is likely to happen [1]. Their main limitations are:

- Information is scattered across sources and is reviewed only after results are announced.
- Analysis is generic; a student does not receive a personalised interpretation of their own data.
- There is no systematic forward-looking estimate or early risk indication.
- Staff workload limits how many students can be counselled individually.
- Raw numbers do not say what to do next, and there is no interactive way to ask questions about them.

2.2 Problems Addressed by the Proposed System

EduAssist AI centralises the analysis behind one interface, estimates a performance score, classifies academic risk, generates recommendations and study guidance, and lets students ask questions within the limits of a keyword-based assistant.

2.3 Functional Requirements

Table 1: Functional requirements

ID	Requirement	Related endpoint
FR1	Provide a service status response	GET /
FR2	Report whether the models are loaded and ready	GET /model-status
FR3	Predict a performance score	POST /predict/performance
FR4	Classify academic risk	POST /predict/risk
FR5	Retrieve a student's academic data	GET /student/{student_id}
FR6	Generate Student Intelligence	GET /student/{student_id}/intelligence
FR7	Generate Academic Insights	GET /student/{student_id}/insights
FR8	Answer questions with intent and confidence	POST /chat
FR9	Present all results in a web dashboard	React front end

2.4 Non-Functional Requirements

The system shall be usable (a student identifier is the main input and all results appear on one dashboard), maintainable (front end, API layer, data processing, and models are separate modules), interoperable (components communicate through documented REST/JSON interfaces), transparent (outputs are presented as estimates and the assistant is not presented as generative AI), and reliable (invalid identifiers or inputs return an error response).

2.5 Feasibility Analysis

Technical: React, Vite, FastAPI, Pandas, and scikit-learn are mature, open-source, and well documented. **Economic:** no licensed software is needed and the system runs on an ordinary computer; cloud hosting costs

CHAPTER 3: PROPOSED SYSTEM AND ARCHITECTURE

3.1 Proposed Solution Overview

The proposed solution separates presentation from computation. The React front end handles input and display; the FastAPI backend handles data retrieval, model inference, insight generation, and chat processing, and returns JSON. This separation follows the REST architectural style [8] and makes the system suitable for hosting on cloud infrastructure [7].

3.2 System Architecture

Figure 1 shows the overall architecture. The user interacts with the React front end, which sends HTTP requests to the backend's route layer. The backend reads processed student data, passes prepared features to the models, and composes Student Intelligence and Academic Insights. The keyword-based chat component is part of the backend and uses the same data and insights. The processed CSV datasets and the saved model artifacts (Joblib) are shown as supporting stores.

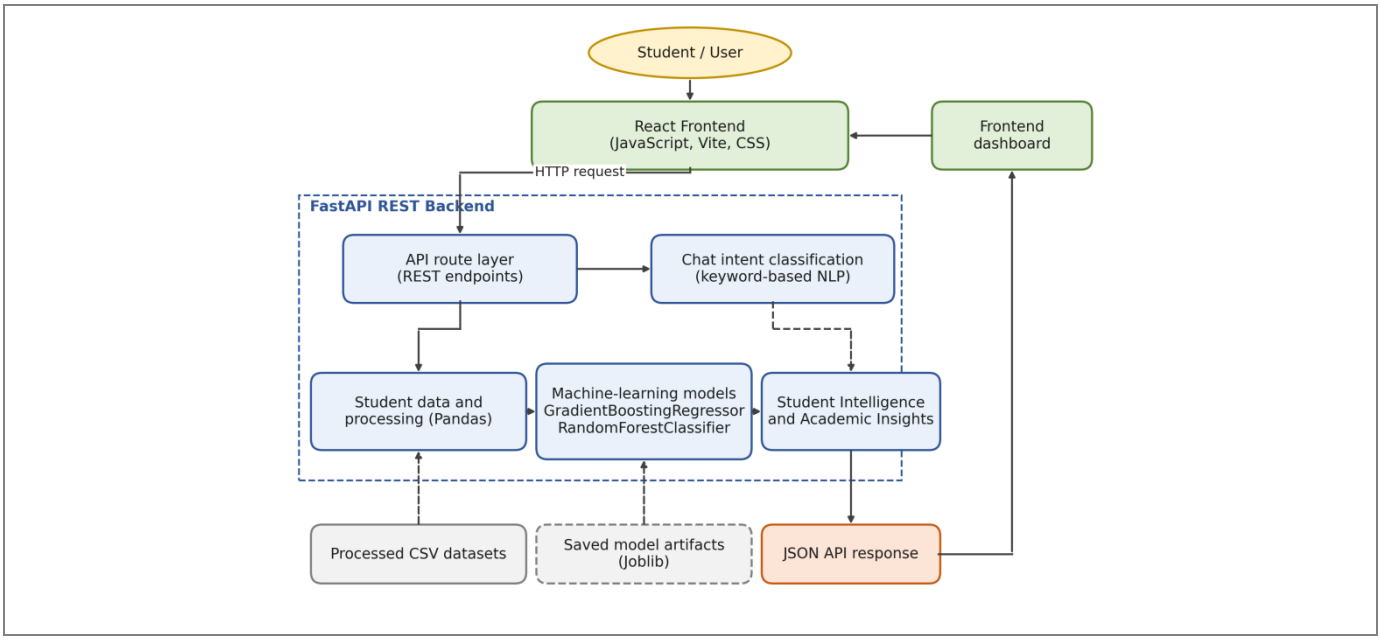


Figure 1: Overall system architecture of EduAssist AI

3.3 Frontend-Backend Interaction and API Communication

Communication is request-response over HTTP using JSON. The front end calls the student data, intelligence, and insights endpoints for a student identifier, the prediction endpoints for model outputs, and the chat endpoint for questions. Figure 2 shows the interaction sequence for the supplied endpoints.

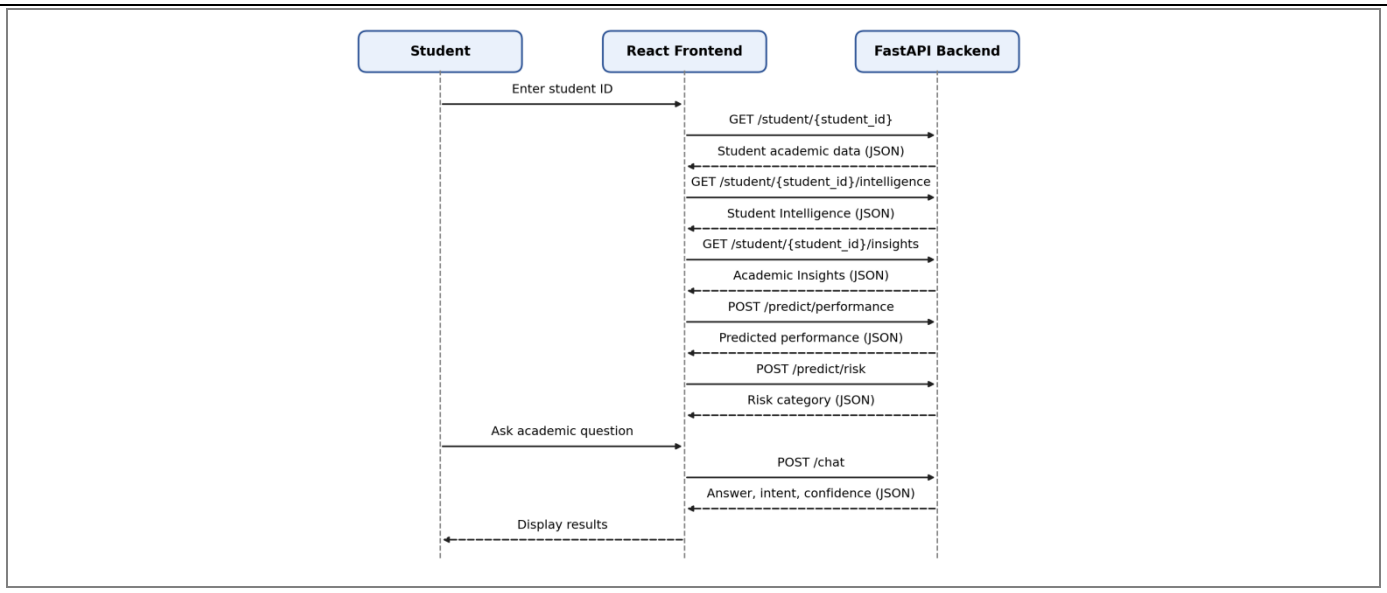


Figure 2: API interaction sequence between the React front end and the FastAPI backend

3.4 Data Flow

Figure 3 shows the end-to-end journey: the student enters an identifier, the front end requests the student's data, the backend returns an error for an unknown identifier or proceeds to analysis, results are displayed on the dashboard, and the student may then ask questions in the chat interface.

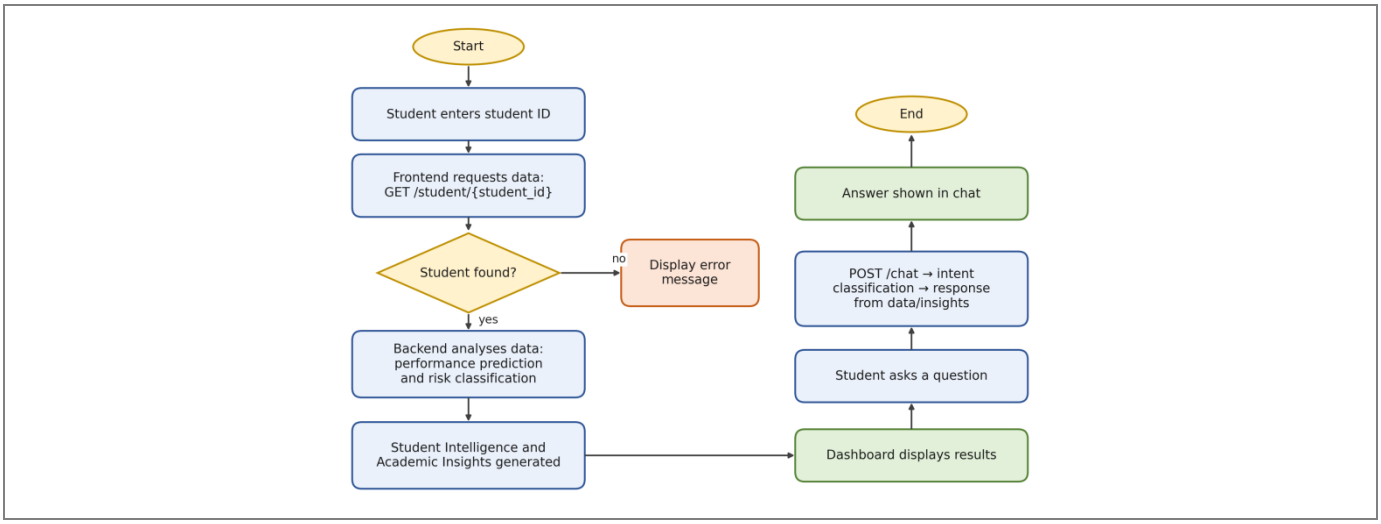


Figure 3: End-to-end system workflow

3.5 Machine-Learning Inference

Two models support the system: a Gradient Boosting Regressor that estimates a performance score and a Random Forest Classifier that assigns an academic risk category. At request time the backend prepares the input features in the order used for training and calls the model's prediction function. Training is a separate stage from runtime inference (Section 5.2).

3.6 Academic Insights Generation

Student Intelligence and Academic Insights combine the student's data, such as average score, performance category, number of assessments, and engagement score, with the risk class to produce summaries, recommendations, and study guidance for the student.

3.7 Chat Processing and Response Generation

A question sent to the chat endpoint is classified into an intent by keyword matching, and the answer is generated from the student's data and insights. The response reports the identified intent and an intent

CHAPTER 4: TECHNOLOGIES AND ALGORITHMS

4.1 Technology Stack

Table 2: Technologies and their roles in EduAssist AI

Technology	Layer	Role in this project
React [4]	Frontend	Component-based interface for the dashboard and chat
JavaScript	Frontend	Front-end logic; API calls and state handling
Vite [5]	Frontend	Development server and build tool for the React application
CSS	Frontend	Styling and layout
Python	Backend	Language for the API, data processing, and models
FastAPI [3]	Backend	REST endpoints, request validation, automatic API documentation
Pandas [6]	Data	Loading and processing the CSV student data
scikit-learn [2]	ML	Gradient Boosting Regressor and Random Forest Classifier
Joblib [10]	ML	Saving and loading trained models, where used
Swagger / OpenAPI [9]	Documentation	Interactive documentation generated by FastAPI

4.2 Frontend and Backend Technologies

React builds the interface from reusable components that re-render when API data changes, which suits a dashboard with several panels. Vite gives fast development builds and CSS defines presentation. Python is used for both the web service and the machine-learning code, so models are called directly from the endpoints. FastAPI validates inputs from type declarations and generates OpenAPI documentation automatically, which supports testing.

4.3 Gradient Boosting Regression

Gradient boosting builds an additive model in stages, each new decision tree being fitted to correct the residual errors of the ensemble so far [11]. In scikit-learn it is provided as GradientBoostingRegressor [2]. It is used here to estimate a numerical performance score because performance is a continuous quantity and boosted trees capture non-linear relationships in tabular data without feature scaling.

4.4 Random Forest Classification

A Random Forest is an ensemble of decision trees, each trained on a random sample of the data and a random subset of features, whose votes are combined to give the class [12]. RandomForestClassifier assigns each student an academic risk class. It handles numerical tabular features directly, is relatively robust to over-fitting, and can report class probabilities.

4.5 Data Processing and Model Persistence

Pandas reads the processed CSV datasets into data frames, from which a single student's data is selected and converted into model features. Joblib, where used, serialises trained scikit-learn models to files so that the API can load them instead of retraining at request time.

CHAPTER 5: SYSTEM DESIGN AND METHODOLOGY

5.1 Implemented Workflow

The workflow has eleven steps, summarised in Table 3.

Table 3: Workflow steps

No.	Step	Description
1	Student identification	The student identifier is entered and sent to the backend.
2	Academic data retrieval	The student's record is retrieved from the processed data.
3	Data preprocessing	Data is cleaned and organised with Pandas.
4	Feature preparation	Model inputs are assembled in the order used for training.
5	Performance prediction	The Gradient Boosting Regressor returns a score.
6	Risk classification	The Random Forest Classifier returns a risk class.
7	Student Intelligence	A student-level intelligence summary is generated.
8	Academic Insights	Recommendations and study guidance are generated.
9	Chat intent classification	A question is mapped to an intent with a confidence value.
10	Response generation	An answer is composed from the student's data and insights.
11	Presentation	The front end renders the JSON results.

5.2 Machine-Learning Workflow

Figure 4 separates two stages. In the training stage the processed CSV dataset is loaded, preprocessed, converted into features, used to fit the two models, and the trained models are saved. In the runtime stage the backend loads the saved models, receives input features, prepares them in training order, runs inference, and returns the result as JSON.

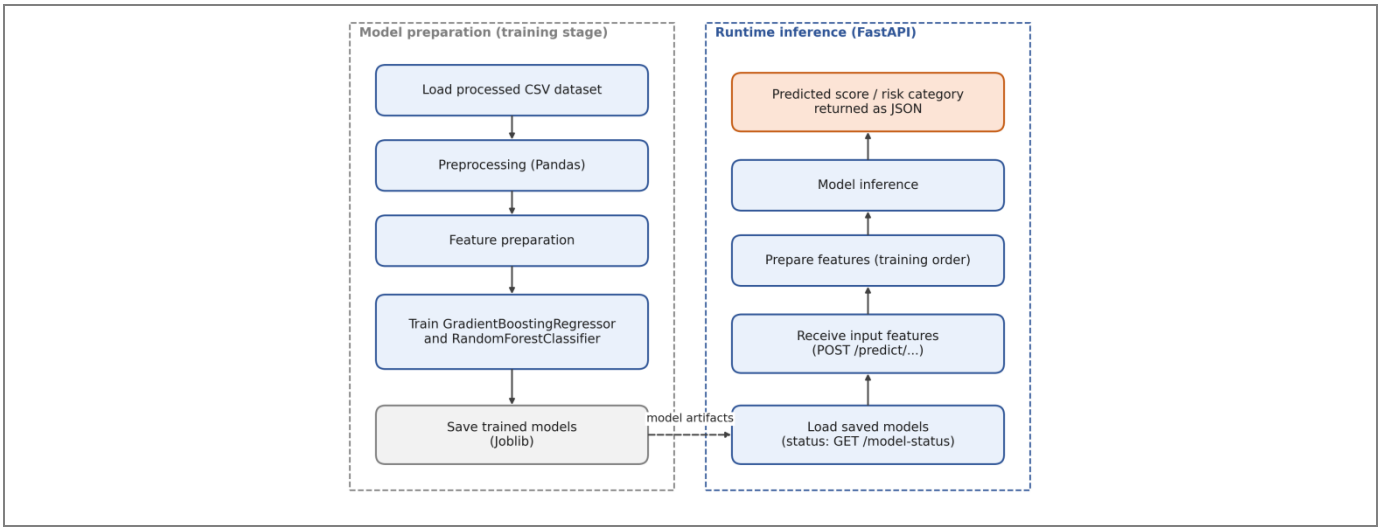


Figure 4: Machine-learning workflow: training stage and runtime inference

5.3 Academic Risk Classification

Figure 5 shows the risk path. Student features are prepared and passed to the Random Forest Classifier, whose predicted risk class is returned by POST /predict/risk and displayed by the front end. In the sample record of Table 7 the returned class is Medium.

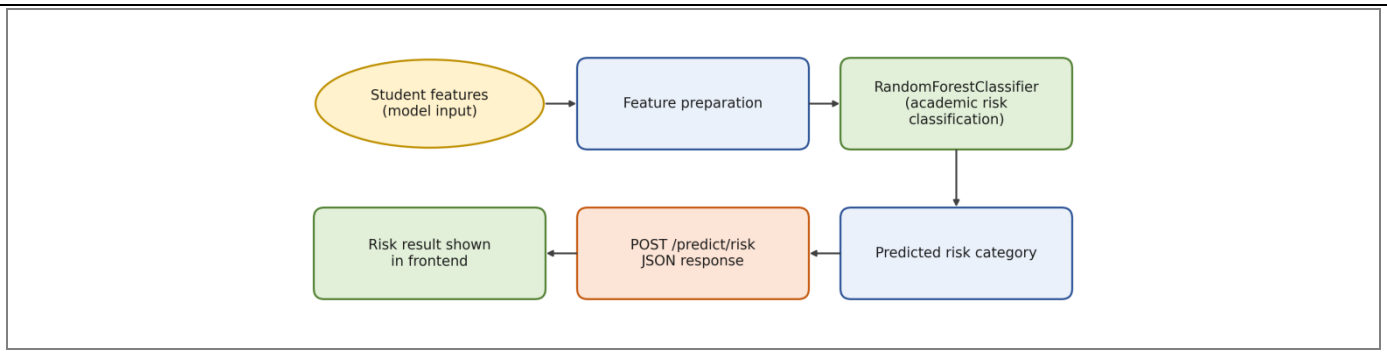


Figure 5: Academic risk classification workflow

5.4 Performance Prediction

Figure 6 shows the performance path. The Gradient Boosting Regressor produces a numerical score that is returned by POST /predict/performance and displayed by the front end.

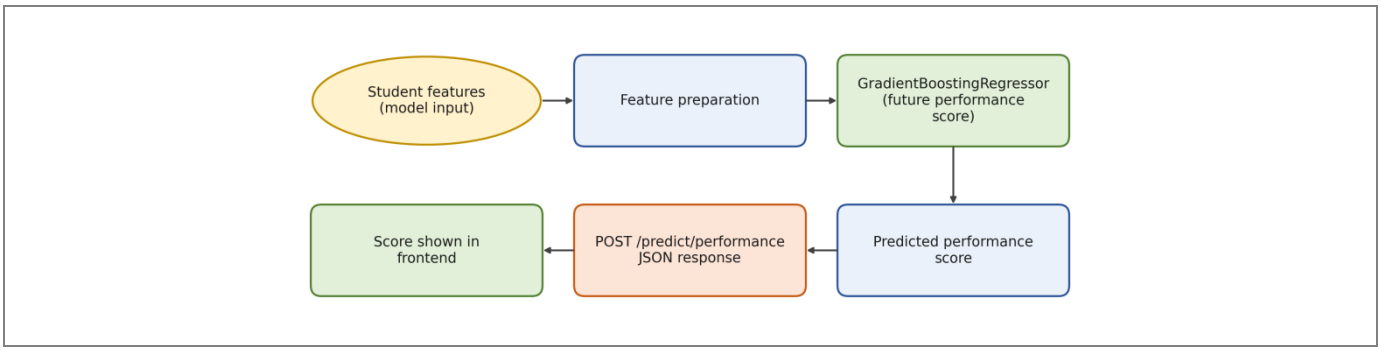


Figure 6: Performance prediction workflow

5.5 Chat Assistant Workflow

Figure 7 shows how a question is processed. The chat endpoint classifies the intent by keywords, retrieves the relevant student data or insights, generates a rule-based response, and returns it as JSON in the answer field for display in the chat interface.

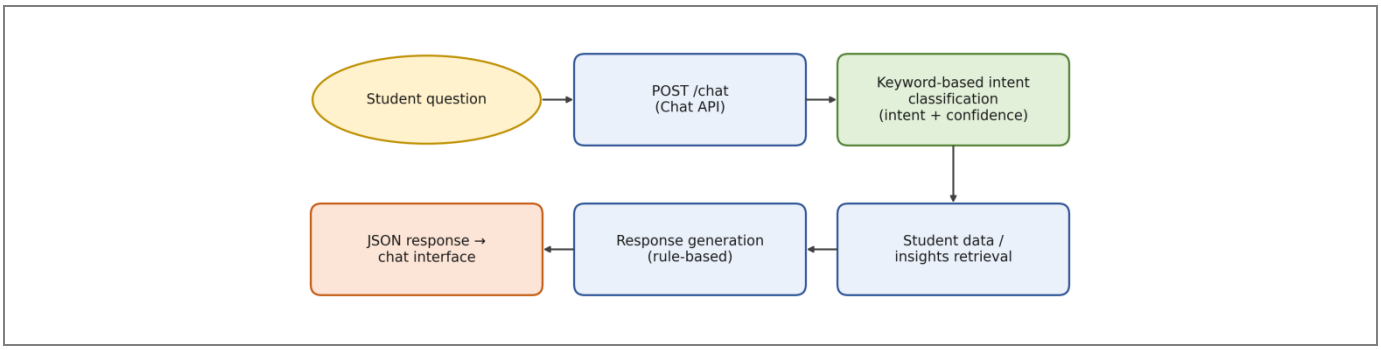


Figure 7: Chat assistant workflow

CHAPTER 6: IMPLEMENTATION

6.1 Frontend and Backend Structure

The front end is a React application built with Vite. It provides an input for the student identifier, panels for the student overview, predictions, risk, Student Intelligence, and Academic Insights, and a chat interface. The backend is a FastAPI application that defines the endpoints of Table 5. Behind the route layer are the student data processing, the model loading and prediction code, the insight generation, and the chat assistant, as shown in Figure 8.

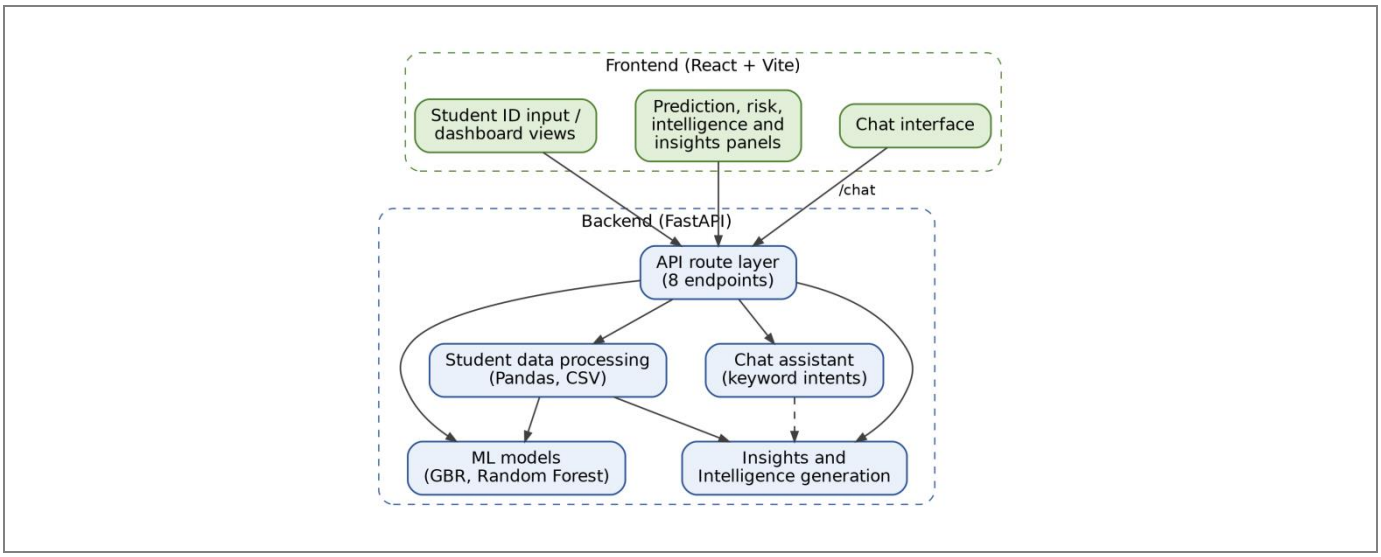


Figure 8: Project module and component diagram

6.2 Components and Responsibilities

Table 4: Project components and responsibilities

Component	Responsibility
Frontend application (React, Vite, CSS)	Student identifier input; dashboard panels; chat interface; calls to the backend
Backend API (FastAPI)	Route definitions, request validation, JSON responses, API documentation
Student data processing (Pandas, CSV)	Reads processed student data and prepares records and model features
Performance model (GradientBoostingRegressor)	Estimates the performance score
Risk model (RandomForestClassifier)	Assigns the academic risk class
Model persistence (Joblib)	Saves and loads trained model artifacts, where used
Intelligence and insights generation	Builds Student Intelligence, recommendations, and study guidance
Chat assistant	Keyword-based intent classification and response generation

6.3 API Implementation

Table 5 lists the endpoints. Request and response bodies are exchanged as JSON; the interactive Swagger page generated by FastAPI documents each endpoint.

Table 5: API endpoints

Method	Endpoint	Purpose	Input / response
GET	/	Service status	No input; status message
GET	/model-status	Model readiness	No input; model loading status
POST	/predict/performance	Performance score	JSON body of model features; predicted score
POST	/predict/risk	Risk class	JSON body of model features; predicted risk class
GET	/student/{student_id}	Student data	Path parameter student_id; student record and metrics (average score, category, number of assessments, engagement score)
GET	/student/{student_id}/intelligence	Student Intelligence	Path parameter student_id; intelligence summary
GET	/student/{student_id}/insights	Academic Insights	Path parameter student_id; insights and recommendations
POST	/chat	Chat answer	JSON body with the question; answer, intent, and intent confidence

6.4 Model Loading, Data Processing, Insights, and Chat

Model loading and prediction: the backend obtains the two trained models, prepares the feature vector, and calls the prediction methods; GET /model-status reports readiness. **Data processing:** Pandas reads the processed CSV data and selects the record of the requested student. **Insights:** the insight functions combine data and the risk class into recommendations. **Chat:** a keyword matcher selects an intent and a response builder composes the answer.

6.5 Frontend-Backend Integration, Error Handling, and Validation

The front end calls the endpoints over HTTP and renders the JSON responses. FastAPI validates request data against declared types, and an unknown student identifier or malformed input produces an error response that the interface can report to the user.

CHAPTER 7: TESTING AND RESULTS

7.1 Testing Methodology

Testing has four parts: API testing of each endpoint through the Swagger interface or an HTTP client, frontend integration testing with the backend running, functional testing of the machine-learning outputs, and valid and invalid input testing. Table 6 lists the test cases and the behaviour expected from each.

Table 6: Test cases and expected results

ID	Test	Input	Expected result
TC1	Service status	GET /	Success response
TC2	Model status	GET /model-status	Model readiness reported
TC3	Student data, valid ID	GET /student/{valid id}	Student data returned
TC4	Student data, unknown ID	GET /student/{unknown id}	Error response
TC5	Performance prediction, valid input	POST /predict/performance	Score returned
TC6	Risk classification, valid input	POST /predict/risk	Risk class returned
TC7	Prediction, missing or invalid fields	POST with incomplete body	Validation error
TC8	Intelligence and insights	GET .../intelligence and .../insights	Responses returned
TC9	Chat, recognised question	POST /chat	Answer with intent and confidence
TC10	Chat, unrecognised question	POST /chat	Graceful fallback answer
TC11	Frontend integration	Enter student ID in the interface	All panels populated

7.2 Machine-Learning Functionality Testing

Functional testing checks that the models load, that predictions are returned for valid input, and that outputs fall in the expected range or label set. No statistical evaluation measures (such as accuracy, precision, recall, F1-score, MAE, or R^2), dataset sizes, or response times are reported in this document.

7.3 Recorded Output for a Sample Student

Table 7 shows the output recorded in an earlier test for student ID 11391. It is a single record and is given only to illustrate the form of the output; it is not a measure of model performance.

Table 7: Recorded output for student ID 11391

Item	Recorded value
Average score	About 82%
Performance category	Excellent
Number of assessments	5
Engagement score	About 7.60

CHAPTER 8: USER INTERFACE AND OUTPUT ANALYSIS

8.1 User Interface Design Principles

The interface follows a small set of principles suited to a student-facing academic tool. It is **simple**, since a student identifier is the main input; **consistent**, since each result type is shown in its own labelled panel; **informative**, since predictions are shown with their meaning, for example a risk class together with recommendations; and **forgiving**, since an invalid identifier or a failed request should produce a clear message instead of a blank screen. Results are shown as estimates, so that students read them as guidance and not as final verdicts.

8.2 Dashboard and Student Overview

The dashboard is the entry point. It accepts the student identifier and, once the data is retrieved from GET /student/{student_id}, presents the student overview: the average score, the performance category, the number of assessments, and the engagement score.

8.3 Performance Prediction and Risk Classification

The performance panel displays the score estimated by the Gradient Boosting Regressor, and the risk panel displays the class assigned by the Random Forest Classifier. The risk class is shown as a label with explanatory text because a label is easier to interpret than a number alone.

8.4 Student Intelligence and Academic Insights

The intelligence and insights panels turn numbers into guidance. Student Intelligence summarises the student's position, and Academic Insights lists recommendations and study guidance.

8.5 Chat Assistant Interface

The chat interface lets a student type a question and read the answer in a conversation view. The backend returns the answer together with the identified intent and the intent confidence, which makes the assistant's behaviour transparent. The interface works best with direct questions because intent detection is keyword-based.

8.6 API Documentation and Error Handling

FastAPI provides an interactive Swagger page that lists every endpoint and allows requests to be tried directly, which supports testing and gives developers a live reference. For errors, the interface should tell the user what went wrong, such as an unknown student identifier or missing input, and allow the user to try again.

CHAPTER 9: ADVANTAGES, LIMITATIONS AND APPLICATIONS

9.1 Advantages

- **Student-specific analysis:** results are produced for an individual student, not a class average.
- **Automated performance estimation:** a regression model provides a forward-looking score without manual calculation.
- **Academic risk identification:** a classifier flags risk to support early attention.
- **Centralised insights:** data, predictions, intelligence, and insights appear on one dashboard.
- **Interactive assistance:** students can ask questions about their own data.
- **Modular, API-based design:** front end and backend are independent, easing maintenance and later cloud hosting.

9.2 Limitations

- Predictions are estimates and not guaranteed outcomes; they support academic decision-making and do not replace the judgement of faculty and students.
- Model quality depends on the size and quality of the CSV data, and no evaluation metrics are reported.
- The assistant is keyword-based; it is not a generative or conversational AI system.
- Cloud deployment, authentication, role-based access, and database storage are not part of the current implementation.

9.3 Applications

The system can serve as a student self-service academic dashboard, a tool for faculty mentors to identify students who may need support, a basis for academic counselling, and a teaching example of integrating machine learning into a web application. Institutional use would require data-protection controls, expert validation of the models, and responsible communication of risk classes.

CHAPTER 10: FUTURE SCOPE AND CONCLUSION

10.1 Future Scope

- **Cloud deployment** of the backend and front end on a cloud platform [7], with monitoring.
- **Secure authentication and role-based access** for students, faculty, and administrators.
- **Database integration** to replace CSV files with persistent, queryable storage.
- **Improved NLP and conversational capability** beyond keyword matching, with careful evaluation.
- **Expanded academic datasets** covering more students, courses, and semesters.
- **Model evaluation and retraining** on held-out data with reported metrics.
- **Notifications and progress tracking** for students and mentors.
- **Integration with learning management systems** for automatic data collection.

10.2 Conclusion

EduAssist AI brings student data analysis, performance estimation, academic risk classification, personalised insights, and a keyword-based question-answering assistant into one web application built with React, Vite, FastAPI, Pandas, and scikit-learn. Its main contributions are the modular REST architecture, the use of a Gradient Boosting Regressor and a Random Forest Classifier for student-level analysis, and the integrated dashboard and chat interface.

The project's strengths are its clear separation of concerns and its use of well-established tools. Its limitations are that the assistant is rule-based, predictions are estimates, no evaluation metrics are reported, and security and cloud deployment are future work. With a larger dataset, formal model evaluation, authentication, a database, and cloud deployment, the system could grow into a more complete academic support platform.

REFERENCES

[1] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," IEEE Trans. Syst., Man, Cybern. C, Appl. Rev., vol. 40, no. 6, pp. 601–618, Nov. 2010.

[2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[3] FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com

[4] React documentation. [Online]. Available: https://react.dev

[5] Vite documentation. [Online]. Available: https://vite.dev

[6] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 56–61.

[7] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, Sep. 2011.

[8] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[9] OpenAPI Initiative, "OpenAPI Specification." [Online]. Available: https://spec.openapis.org/oas/latest.html

[10] Joblib documentation. [Online]. Available: https://joblib.readthedocs.io

[11] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[12] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.